

Recall Before You Rank: Similarity-Guided Top- K Reuse for Efficient Long-Context Attention

Wenshuai Yao^{1*}, Wenyong Zhou^{2*}, Hanyong Shao¹, Yizhe Chen³,
Zhiyuan Ning¹, Yuannuo Feng³, Ru Huang^{1†}, Kechao Tang^{1†}

¹School of Integrated Circuits, Peking University, Beijing, China

²Department of Electrical and Computer Engineering, The University of Hong Kong, Hong Kong SAR, China

³School of Integrated Circuit Science and Engineering, Beihang University, Beijing, China

Abstract

Top- K sparse attention reduces the cost of Softmax and value aggregation by attending to only a small subset of key-value (KV) entries. However, identifying this subset still requires scoring the current query against the full KV cache and performing global Top- K selection, leaving selector cost linear in context length and limiting the practical efficiency of sparse attention for long-context decoding. In this paper, we introduce **ReTopK**, a training-free method that accelerates dynamic Top- K attention by reusing historical retrieval decisions. ReTopK builds on the observation that similar queries often attend to overlapping supports and that partially overlapping supports can still preserve most of the Exact Top- K attention mass. For each attention head, it maintains a bounded cache of historical query-support pairs, retrieves the most similar cached queries for each new query, unions their stored supports with a recent window, and reranks only the resulting compact candidate set using exact current-query scores. A similarity-based fallback invokes full-history Exact Top- K when reuse is unreliable, while periodic exact refreshes limit cache drift. ReTopK retains the complete KV cache and reuses only selected indices, rather than historical scores, attention weights, or outputs. Across 16K–128K contexts, ReTopK achieves the lowest PG19 perplexity and the highest NIAH and LongBench scores among the evaluated approximate methods. At 128K with $K = 512$, ReTopK incurs only a 0.50% perplexity increase over Exact Top- K while accelerating attention computation by $3.07\times$.

Introduction

Large language models (LLMs) increasingly process long documents, code repositories, and extended interaction histories. During autoregressive decoding, however, the cost of attention grows with the key-value (KV) cache: each new query must compare against a growing history of keys and aggregate the corresponding values. Although optimized kernels such as FlashAttention (Dao et al. 2022; Dao 2024) improve hardware efficiency, they do not remove the linear dependence of decoding attention on context length.

Top- K sparse attention offers a promising alternative by restricting Softmax and value aggregation to the K highest-

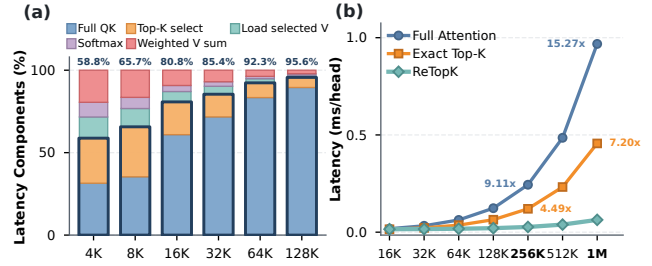


Figure 1: Long-context attention latency: (a) Exact Top- K breakdown at $K = 512$, with full-history QK and Top- K selection outlined; (b) per-head BF16 latency.

scoring KV entries. Yet, conventional Exact Top- K attention must still score the current query against the entire KV cache and perform global Top- K selection before sparse aggregation can begin. Thus, it sparsifies attention aggregation but not index discovery. As shown in Figure 1, full-history query-key (QK) scoring and Top- K selection dominate the latency of Exact Top- K attention, accounting for an increasingly large fraction of runtime as the context grows. Consequently, efficient long-context sparse attention requires reducing not only the cost of attending to selected entries, but also the cost of identifying them.

Existing approaches address this challenge from complementary directions. KV-cache reduction and streaming methods bound attention cost by retaining a limited subset of tokens (Xiao et al. 2024; Zhang et al. 2023), but discarded positions are unavailable if they become relevant later. Query-aware methods preserve more dynamic access to the context through page-level pruning, compressed representations, or approximate retrieval (Tang et al. 2024; Singhania et al. 2024); however, their selection procedures typically still inspect a search space that grows with context length. This leaves an important question: *can a dynamic selector recover query-specific attention supports without searching the full history at every decoding step?*

Our answer is motivated by a simple observation: similar queries often induce overlapping Top- K supports, and even incomplete support overlap can preserve most of the atten-

*These authors contributed equally.

†Corresponding author.

tion mass. Historical query–support pairs therefore provide a compact record of prior retrieval decisions. Rather than recomputing a full-history Top- K support for every query, the selector can first recall supports associated with similar historical queries and then rerank only their union.

In this paper, we introduce **ReTopK**, a training-free method that accelerates dynamic Top- K attention through retrieval-decision reuse. ReTopK maintains a bounded query–support cache independently for each attention head. For a new query, it retrieves the most similar cached queries, unions their stored Top- K supports with a recent-token window, and reranks this compact candidate set using exact current-query scores. The resulting support is then used for sparse Softmax–value aggregation. ReTopK uses historical index selections only as candidates: it does not reuse historical attention scores, weights, or outputs. To avoid unreliable reuse, it falls back to full-history Exact Top- K when cached-query similarity is low and periodically refreshes the cache with exact supports. Our contributions are as follows:

- We identify full-history index discovery as a central bottleneck of Exact Top- K attention and show that query similarity provides an effective signal for reusing sparse attention supports.
- We propose ReTopK, a training-free recall-before-rerank method that combines a per-head query–support cache, similarity-guided candidate construction, exact candidate reranking, similarity-based fallback, and periodic refresh.
- We develop a fused GPU implementation and evaluate ReTopK across multiple models and long-context tasks. At 128K with $K = 512$, ReTopK incurs only a 0.50% perplexity increase over Exact Top- K while accelerating attention computation by $3.07\times$.

Related Work

Efficient attention and Exact Top- K attention. IO-aware attention kernels, including FlashAttention and FlashAttention-2, improve the efficiency of dense attention through tiling, fusion, and reduced memory traffic (Dao et al. 2022; Dao 2024). However, autoregressive decoding still requires each query to interact with a KV cache whose length grows with the context. Exact Top- K attention reduces the cost of Softmax–value aggregation by retaining only the globally highest-scoring K entries, but it must first compute QK scores over the full history and perform global Top- K selection. ReTopK targets this remaining index-discovery cost: it approximates the support-selection process while retaining exact current-query scoring within the recalled candidate set.

KV-cache reduction and structured sparsity. A complementary line of work bounds attention cost by reducing or organizing the active KV cache. Streaming and eviction methods retain attention sinks, recent tokens, or tokens selected by accumulated importance (Xiao et al. 2024; Liu et al. 2023; Zhang et al. 2023). Other approaches compress the cache using observation-window signals, layer-dependent budgets, or head-specific policies (Li et al. 2024; Cai et al. 2025; Xiao et al. 2025). Structured sparse methods further exploit recurring attention patterns to accelerate long-context computation (Jiang et al. 2024). These approaches provide bounded

memory or predictable computation, but may restrict access to positions excluded from the active cache or sparse pattern. In contrast, ReTopK preserves the complete KV cache without eviction, enabling query-dependent retrieval from the full history.

Query-aware sparse selection. Query-aware methods reduce attention cost by estimating relevant context from the current query. SparQ estimates key relevance from selected query dimensions, Quest prunes KV pages with query-dependent bounds, and Loki searches a low-rank key space (Ribar et al. 2024; Tang et al. 2024; Singhania et al. 2024). RetrievalAttention indexes KV entries, Squeezed Attention retrieves key clusters, and SeerAttention predicts sparse structures with learned gates (Liu et al. 2025; Hooper et al. 2025; Gao et al. 2025). TokenSelect caches a selection for similar consecutive queries (Wu et al. 2025). Recent methods reuse sparse selections across layers or decoding steps: Cascade and IndexCache share Top- K indices across layers, while PRR overlaps temporal reuse with exact selection and repairs missed blocks (Deshmukh et al. 2025; Bai et al. 2026; Wang et al. 2026). ReTopK instead retrieves multiple supports from a bounded per-head query cache and reranks their union with current-query scores, avoiding a full-history scan on reuse steps; fallback and refresh maintain reliability without training or KV eviction.

Methodology

Motivation: Query Similarity and Support Reuse

We begin by examining whether historical attention decisions can help identify the important positions for a new query. We consider one attention layer and one query head, and omit these indices throughout this section. Let q_t denote the query at decoding step t , and let $\{k_i\}_{i=1}^{L_t}$ be the keys currently available in the KV cache. The Exact Top- K support is

$$S_t = \text{TopK}_{i \in \{1, \dots, L_t\}} \left(\frac{q_t^\top k_i}{\sqrt{d}}, K \right), \quad (1)$$

where S_t contains the indices of the K largest scores.

For two queries at positions t and j , we measure their normalized support overlap as

$$\text{Overlap}(t, j) = \frac{|S_t \cap S_j|}{K}. \quad (2)$$

Given the R most similar preceding queries $\{j_1, \dots, j_R\}$, we further define the union of their supports and its coverage of the current Exact Top- K support as

$$U_t(R) = \bigcup_{r=1}^R S_{j_r}, \quad \text{Coverage}_t(R) = \frac{|S_t \cap U_t(R)|}{K}. \quad (3)$$

Figure 2 shows that query cosine similarity is strongly correlated with Exact Top- K support overlap. This relation holds both across historical query pairs and when each query is matched to its most similar preceding query. Although the support of a single historical query is incomplete, unioning supports from multiple similar queries substantially improves coverage.

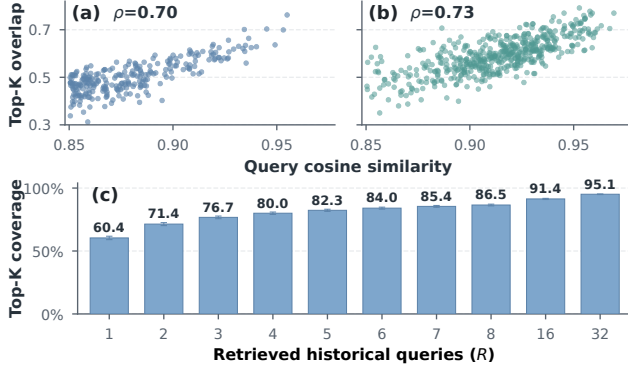


Figure 2: Query similarity versus Exact Top- K support overlap: (a) all historical pairs; (b) each query’s most similar predecessor; (c) coverage by the union of R retrieved supports.

Reuse quality is not uniform across attention heads. As shown in Figure 3, some heads achieve near-complete coverage from retrieved supports, whereas others exhibit substantially lower coverage. Within each head, however, coverage remains relatively stable over consecutive decoding steps. This head-dependent behavior motivates maintaining retrieval decisions independently for each query head. It also shows that reuse should not be forced universally: a practical method requires a mechanism to identify and recover from unreliable reuse.

These observations lead to three design principles. First, retrieval decisions should be stored together with their originating queries, so that query similarity can serve as a lightweight retrieval signal. Second, recalled supports should be treated as candidates rather than final selections, and should be reranked under the current query. Third, reuse must be paired with safeguards for low-confidence and stale retrieval decisions. ReTopK implements these principles through a bounded per-head query-support cache, exact reranking over recalled candidates, similarity-based fallback, and periodic refresh.

Problem Setup and ReTopK Overview

At decoding step t , let $q_t \in \mathbb{R}^d$ denote the current query, and let $\{(k_i, v_i)\}_{i=1}^{L_t}$ denote the KV cache. Exact Top- K attention computes a score for every cached key,

$$e_{t,i} = \frac{q_t^\top k_i}{\sqrt{d}}, \quad i \in \{1, \dots, L_t\}, \quad (4)$$

selects the full-history support S_t in Equation 1, and performs Softmax-value aggregation over the selected entries. Although this aggregation involves only K entries, obtaining S_t still requires full-history QK scoring and global Top- K selection.

On reuse steps, ReTopK replaces this full-history selection with a *recall-before-rerank* procedure. It first retrieves historical supports associated with queries similar to q_t , then constructs a compact candidate set from these supports and a recent window. ReTopK computes exact current-query scores

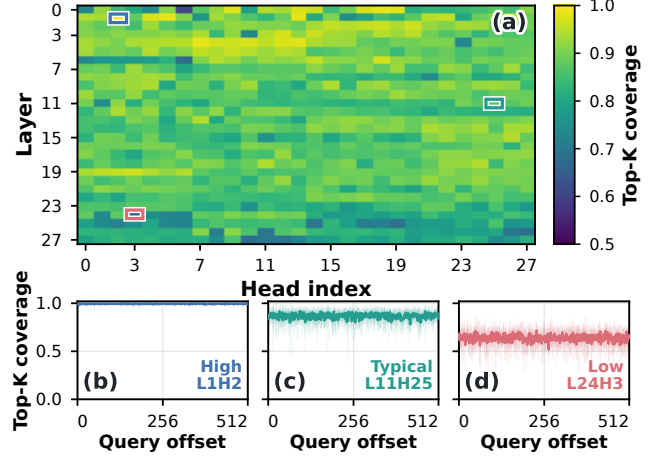


Figure 3: Head-wise Exact Top- K support coverage on Qwen2.5-7B at 16K ($K = 512$, $R = 8$): (a) layer-head means; (b–d) representative decoding traces.

only for the candidate keys and selects the final Top- K entries within this set. When reuse is unreliable, it reverts to Exact Top- K attention.

Figure 4 compares this procedure with Exact Top- K attention. Exact Top- K scores all L_t keys at every step, whereas ReTopK searches only a bounded cache of historical queries and reranks $M_t \ll L_t$ candidate keys on the reuse path. Importantly, ReTopK reuses only historical *index selections*; it does not reuse historical QK scores, attention probabilities, or attention outputs.

Recall-Before-Rerank Selection

Query-support cache. For each query head, ReTopK maintains a bounded FIFO cache of capacity C :

$$\mathcal{B}_t = \left\{ \left(\bar{q}_c, \hat{S}_c \right) \right\}_{c=1}^{|\mathcal{B}_t|}, \quad \bar{q}_c = \frac{q_c}{\|q_c\|_2}, \quad (5)$$

where $\bar{q}_c$ is the normalized query representation after positional encoding, and $\hat{S}_c$ is the support selected at the corresponding historical step. The cache is initialized from the final C prompt tokens during prefill and updated after each decoding step, whether ReTopK takes the reuse or exact path. The cache adds bounded metadata to the original KV cache: it stores normalized queries and selected indices, while preserving the complete set of keys and values.

Similarity-guided recall. For the normalized current query $\bar{q}_t = q_t / \|q_t\|_2$, ReTopK computes cosine similarities to all cached queries:

$$\rho_{t,c} = \bar{q}_t^\top \bar{q}_c, \quad c \in \{1, \dots, |\mathcal{B}_t|\}. \quad (6)$$

It retrieves the indices of the R most similar cache entries,

$$\mathcal{R}_t = \text{TopR}_{c \in \{1, \dots, |\mathcal{B}_t|\}}(\rho_{t,c}, R), \quad \rho_t^{\max} = \max_c \rho_{t,c}. \quad (7)$$

Unlike Exact Top- K , this recall stage compares q_t with only C cached queries rather than all L_t historical keys.

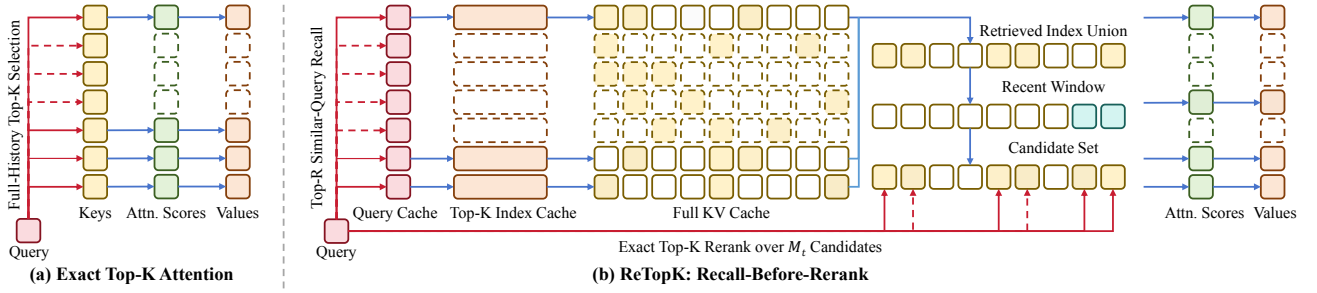


Figure 4: (a) Full-history Exact Top- K selection. (b) ReTopK recall, candidate construction, and reranking. Dashed boxes and lines denote unselected entries and connections.

Candidate construction and reranking. The supports associated with the retrieved cache entries are merged with a recent-token window,

$$\mathcal{L}_t = \{\max(1, L_t - W + 1), \dots, L_t\}, \quad (8)$$

where W is the window size. The resulting candidate set is

$$\mathcal{A}_t = \text{Unique} \left(\bigcup_{c \in \mathcal{R}_t} \hat{S}_c \cup \mathcal{L}_t \right), \quad M_t = |\mathcal{A}_t|. \quad (9)$$

The recent window ensures that newly appended tokens remain eligible even if they do not appear in any retrieved support.

ReTopK then computes exact current-query QK scores only for candidate keys:

$$\hat{e}_{t,i} = \frac{q_t^\top k_i}{\sqrt{d}}, \quad i \in \mathcal{A}_t, \quad (10)$$

and selects the reranked support

$$\hat{S}_t = \text{TopK}_{i \in \mathcal{A}_t}(\hat{e}_{t,i}, K). \quad (11)$$

Reranking maps the variable-size candidate set back to exactly K indices, preserving the fixed-size support stored in each cache entry. If $S_t \subseteq \mathcal{A}_t$, this reranking step exactly recovers S_t .

Finally, ReTopK performs sparse attention over the reranked support:

$$\hat{o}_t = \sum_{i \in \hat{S}_t} \frac{\exp(\hat{e}_{t,i})}{\sum_{j \in \hat{S}_t} \exp(\hat{e}_{t,j})} v_i. \quad (12)$$

Thus, historical supports are used only for candidate generation. All retained scores, normalized attention weights, and value aggregations are computed using the current query and the original KV cache.

Reliable Reuse via Fallback and Refresh

Query similarity provides a useful but imperfect signal for reuse fidelity. ReTopK therefore takes the Exact Top- K path whenever the maximum cached-query similarity falls below a threshold τ , i.e., $\rho_t^{\max} < \tau$. This fallback avoids reliance on historical supports when no sufficiently related query is available.

Fallback alone does not fully address error accumulation, because supports produced on the reuse path are subsequently inserted into the cache. To periodically inject exact retrieval decisions, ReTopK performs an Exact Top- K refresh every T_r decoding steps; setting $T_r = 0$ disables scheduled refresh. Let t_{dec} denote the decoding-step offset and $\mathcal{P}_t$ the selected path. Then

$$\mathcal{P}_t = \begin{cases} \text{EXACT}, & T_r > 0, t_{\text{dec}} \in \{T_r, 2T_r, \dots\}, \\ \text{EXACT}, & \rho_t^{\max} < \tau, \\ \text{REUSE}, & \text{otherwise.} \end{cases} \quad (13)$$

The two mechanisms are complementary: similarity fallback handles queries without reliable cached-query matches, whereas periodic refresh limits long-horizon drift in the on-line cache.

After either path, ReTopK caches $(\bar{q}_t, \hat{S}_t)$, with $\hat{S}_t = S_t$ on exact steps.

Complexity and GPU Implementation

For Exact Top- K , full-history QK scoring requires $\mathcal{O}(L_t d)$ work, followed by global Top- K selection over L_t scores. In contrast, a ReTopK reuse step requires $\mathcal{O}(Cd)$ work for cache matching, $\mathcal{O}(RK + W)$ work to construct the raw candidate list, and $\mathcal{O}(M_t d)$ work for candidate QK scoring, followed by Top- K selection over M_t scores, where

$$M_t \leq RK + W \ll L_t. \quad (14)$$

The subsequent sparse Softmax-value aggregation remains $\mathcal{O}(K d_v)$, where d_v is the value-head dimension. Hence, with C , R , K , and W fixed, a reuse step replaces both full-history QK scoring and global selection over L_t entries with bounded cache lookup and candidate processing over M_t entries. Its selector work therefore does not grow with context length. The additional metadata cost is $\mathcal{O}(Cd + CK)$ per head, while the complete KV cache remains unchanged.

Our GPU implementation maps the reuse path to three GPU stages. A single fused selector performs cache matching, Top- R retrieval, and exact candidate sorting and deduplication. An indexed QK kernel directly scores the packed candidate keys without materializing a gathered key tensor. A final fused kernel performs candidate Top- K selection, indexed Softmax-value aggregation, and cache update. Cache state and per-layer workspaces are preallocated and kept on the GPU. Exact, fallback, and refresh paths use matched full-history kernels, so the paths differ only in support discovery.

Method	PG19 PPL ↓				Speedup vs. Exact Top- K ↑				NIAH: 3-task Avg. ↑				LongBench ↑			
	16K	32K	64K	128K	16K	32K	64K	128K	16K	32K	64K	128K	2Wiki	Hotpot	MFQA	Avg.
Full Attention	8.78	8.76	8.82	11.62	2.08×	1.18×	0.95×	0.87×	99.5	97.3	98.5	93.2	55.1	59.8	49.8	54.9
Exact Top- K	8.92	8.94	9.03	12.07	1.00×	1.00×	1.00×	1.00×	95.2	95.0	98.0	64.5	49.8	57.5	50.2	52.5
StreamingLLM	11.60	11.49	12.18	12.38	2.19×	2.36×	3.61×	6.39×	3.3	3.8	2.5	3.5	22.3	30.8	24.4	25.8
Quest	10.55	17.05	18.17	36.23	1.07×	1.13×	1.75×	<u>3.09×</u>	43.7	78.8	29.7	24.0	43.0	39.5	42.0	41.5
SparQ	11.87	13.43	13.55	23.26	0.68×	0.51×	0.78×	1.39×	59.2	84.8	48.0	45.3	<u>47.4</u>	52.9	48.8	49.7
Loki ($r = 64$)	61.01	33.28	110.08	199.32	0.57×	0.60×	0.93×	1.37×	4.7	0.0	0.0	0.0	40.4	39.5	46.9	42.3
TokenSelect	10.25	10.60	11.12	17.71	0.67×	0.63×	0.97×	1.33×	65.8	83.3	36.7	44.8	47.3	56.0	49.8	<u>51.0</u>
ReTopK ($\tau = 0.85$)	<u>8.98</u>	<u>9.03</u>	<u>9.18</u>	12.13	<u>1.29×</u>	<u>1.38×</u>	<u>2.03×</u>	<u>3.07×</u>	<u>85.3</u>	<u>89.3</u>	<u>93.2</u>	<u>53.3</u>	47.0	54.5	48.9	50.1
ReTopK ($\tau = 0.90$)	8.95	8.98	9.11	<u>12.20</u>	1.14×	1.23×	1.70×	2.25×	96.3	92.2	98.0	75.2	48.4	58.7	51.0	52.7

Table 1: Long-context performance and attention efficiency ($K = 512$). Speedup is relative to Exact Top- K . Gray and teal denote reference and ReTopK rows, respectively; boldface and underlining mark the best and second-best approximate results.

Experiments

Experimental Setup

Models and benchmarks. We evaluate Qwen2.5-7B (Qwen et al. 2024) on PG19 (Rae et al. 2020) and Qwen2.5-7B-Instruct-1M (Yang et al. 2025) on RULER NIAH (Hsieh et al. 2024) and LongBench (Bai et al. 2024). PG19 and NIAH are evaluated at 16K, 32K, 64K, and 128K contexts. For PG19, we report perplexity over a 512-token teacher-forced suffix for each of 64 fixed documents. For NIAH, we report the macro-average of single-needle, multi-key, and multi-value retrieval over 50 examples per task and context length. For LongBench, we use the complete test splits of three subsets: 2WikiMQA, HotpotQA, and Multi-FieldQA, comprising 550 examples in total.

Baselines. We compare ReTopK with Full Attention, Exact Top- K , StreamingLLM, Quest, SparQ, Loki, and TokenSelect (Xiao et al. 2024; Tang et al. 2024; Ribar et al. 2024; Singhania et al. 2024; Wu et al. 2025). Unless otherwise specified, all sparse methods use a shared $K = 512$ active-token budget. StreamingLLM allocates this budget to 64 sink and 448 recent tokens, whereas TokenSelect uses 64 initial, 416 selected, and 32 local tokens. Other baseline configurations are provided in the appendix.

Implementation settings. Unless otherwise specified, ReTopK uses $C = W = 32$, $R = 4$, $\tau = 0.85$, and $T_r = 128$. All experiments use BF16. Attention latency is measured on a single NVIDIA L20 GPU using matched kernels, and speedup is reported relative to Exact Top- K .

Experimental Results

Overall quality and efficiency. Table 1 reports long-context performance and attention efficiency. Across all context lengths, the two ReTopK settings rank first and second among approximate methods in PG19 PPL and jointly attain the best approximate results across all reported task metrics, while matching or occasionally exceeding Exact Top- K . The default $\tau = 0.85$ achieves a $3.07\times$ speedup at 128K with a 0.50% PPL increase, whereas the quality-oriented $\tau = 0.90$ maintains a 1.14 – $2.25\times$ speedup, leads all approximate methods on NIAH at every length, and obtains a

Model	Ctx.	K	PPL		Δ PPL (%)	Reuse (%)	Speedup vs. Exact
			Exact	ReTopK			
Qwen2.5-7B	16K	512	8.92	8.98	+0.70	87.5	$1.29\times$
	32K	512	8.94	9.03	+1.08	87.7	$1.38\times$
	64K	512	9.03	9.18	+1.60	88.4	$2.03\times$
	128K	1024	11.72	12.05	+2.76	89.2	$1.76\times$
Llama-3.1-8B	16K	512	7.93	8.06	+1.71	86.3	$1.29\times$
	32K	512	8.13	8.29	+2.00	86.1	$1.83\times$
	64K	512	8.15	8.37	+2.62	85.9	$2.66\times$
	128K	1024	7.93	8.11	+2.28	89.6	$2.59\times$
Qwen2.5-14B	16K	512	7.69	7.74	+0.55	82.5	$1.26\times$
	32K	512	7.72	7.74	+0.25	82.8	$1.81\times$
	64K	512	7.74	7.76	+0.21	83.0	$2.59\times$
	128K	1024	8.38	8.34	−0.46	86.3	$2.52\times$

Table 2: Cross-model PG19 results (64 documents; shared ReTopK settings; reuse weighted over head–token pairs).

LongBench average of 52.7, exceeding the 52.5 Exact Top- K reference.

Speedup should be interpreted jointly with task performance. StreamingLLM is fastest, but its fixed sink-and-recent window excludes evidence outside the retained tokens, limiting its NIAH score to 2.5–3.8. TokenSelect reaches 51.0 on LongBench, yet its selection cache hits on only 34.7–35.7% of NIAH steps, restricting its speedup to 0.67–1.33 $\times$. Loki uses the shared $K = 512$ budget and rank 64, substantially below the 12.5–25% context-proportional budgets used in its paper; its results therefore characterize a strict-budget operating point. Paper-recommended configurations are compared in the appendix.

Cross-model generalization. We apply the configuration selected on Qwen2.5-7B to Llama-3.1-8B (Grattafiori et al. 2024) and Qwen2.5-14B without model-specific tuning, using $C = W = 32$, $R = 4$, $\tau = 0.85$, and $T_r = 128$, with $K = 512$ at 16K–64K and $K = 1024$ at 128K. Across all model–context pairs, ReTopK achieves 82.5–89.6% reuse, PPL changes of −0.46% to +2.76% relative to Exact Top-

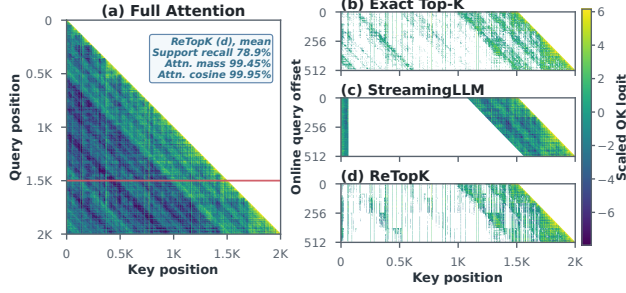


Figure 5: QK supports for Qwen2.5-7B layer 18, head 6 after a 1,536-token prefill ($K = 512$; 512 decoding steps). White denotes unselected or masked entries.

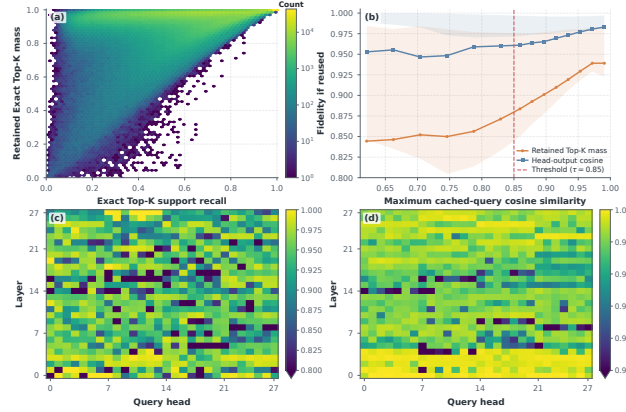


Figure 6: All-head ReTopK fidelity on Qwen2.5-7B for 4 PG19 documents (32K prefill, 512 decoding steps, $K = 512$).

K , and speedups of 1.26–2.66 $\times$, demonstrating consistent transfer across model families and scales.

Attention-support fidelity. ReTopK need not reproduce every index selected by Exact Top- K , provided that the reranked support preserves high-mass positions and the resulting attention output. We evaluate this criterion for both a representative head and all layers and query heads.

Figure 5 compares Full Attention, Exact Top- K , StreamingLLM, and ReTopK for the representative head. ReTopK’s long-range support geometry aligns more closely with Exact Top- K than StreamingLLM’s fixed sink-and-recent pattern. With 78.9% Exact Top- K support recall, ReTopK retains 99.45% of the attention mass and achieves 99.95% attention-distribution cosine, indicating that the mismatched indices receive low normalized attention weights.

Figure 6 extends the analysis to 1,605,632 head-token pairs across all 28 layers and 28 query heads. Retained mass and head-output cosine average 92.4% and 97.4% over all pairs, and 91.6% and 97.2% on reuse-path pairs despite 56.9% support recall. Both metrics increase with cached-query similarity, supporting the similarity fallback at $\tau = 0.85$; the layer-head maps further show that this behavior is distributed throughout the model.

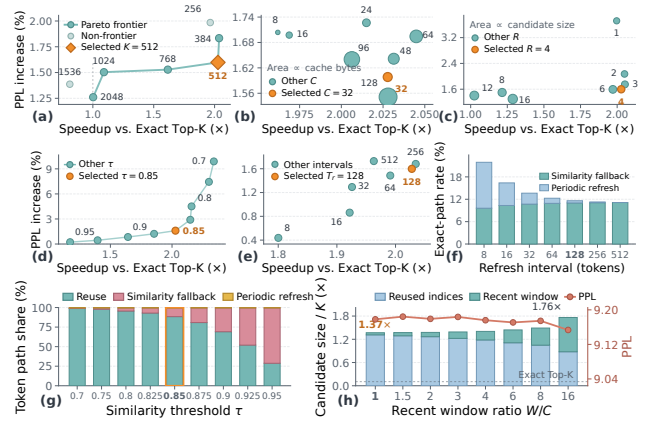


Figure 7: One-at-a-time ReTopK hyperparameter sensitivity at 64K; orange marks defaults.

Ablation	Setting	PPL ↓	Exact %	Cand.	Spd. $\times$
Exact Top- K	Reference	9.034	100.0	–	1.00
ReTopK (default)	Default	9.178	11.6	701	2.03
Cached-query retrieval	Recent R	9.184	16.7	649	1.84
Multi-query union	$R = 1$	9.370	12.1	517	2.00
Historical-query search	$C = 1$	9.250	22.6	515	1.83
Similarity fallback	$\tau = -1$	10.076	0.6	654	2.36
Scheduled refresh	$T_r = 0$	9.190	11.1	689	2.03
Recent-token guard	$W = 0$	716.0	8.0	621	2.15

Table 3: ReTopK component ablation at 64K ($K = 512$).

Hyperparameter sensitivity and component ablation.

Figure 7 evaluates one-at-a-time sensitivity around the default 64K configuration. In panels (a–e), the default $K = 512$ lies on the Pareto frontier, with a 2.03 $\times$ speedup and a 1.60% PPL increase over Exact Top- K . Larger K yields gradual PPL improvements at higher reranking cost, reducing speedup to 1.09 $\times$ at $K = 1024$. Performance is stable near the default C , while gains from increasing R diminish beyond $R = 4$. Increasing τ trades speed for quality by triggering more exact fallbacks. Refresh intervals $T_r \geq 128$ give similar short-horizon PPL, whereas shorter intervals improve quality through more frequent Exact Top- K steps.

Panels (f–h) decompose these trade-offs: increasing T_r reduces the refresh share, while increasing τ shifts decisions from reuse to similarity fallback. Increasing W/C from 1 to 16 enlarges the candidate set from approximately 701 to 901 candidates with little PPL variation; $W = 0$ is examined in the component ablation.

Table 3 isolates the components; Exact and Cand. denote the exact-path rate and mean candidate count. Although recent- R retrieval raises the exact-path rate from 11.6% to 16.7%, it has slightly higher PPL and lower speedup than similarity-guided retrieval. Smaller R or C degrades PPL or speed, while removing similarity fallback trades PPL for speed. Eliminating the recent-token guard causes severe degradation. Scheduled refresh has little short-horizon effect,

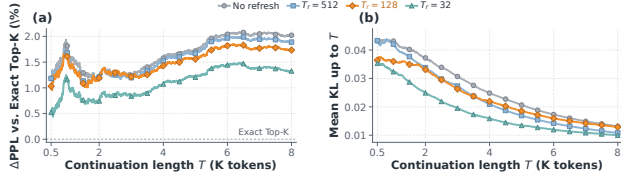


Figure 8: Long-horizon stability under (a) teacher forcing and (b) autoregressive generation.

Level	Optimization stage	64K		128K	
		Lat.	vs. L1	Lat.	vs. L1
L1	Separate-kernel baseline	6.630	1.00×	7.345	1.00×
L2	Candidate discovery and scoring	4.566	1.45×	5.230	1.40×
L3	Candidate selection and output	3.831	1.73×	4.486	1.64×
L4	Persistent workspaces	3.271	2.03×	3.886	1.89×
L5	Candidate-to-output fusion	2.577	2.57×	3.207	2.29×

Table 4: ReTopK kernel-fusion stages at 64K and 128K ($K = 512$; ms/token over 28 layers).

motivating the long-horizon evaluation.

Long-horizon stability. After a 64K prefill, we evaluate 8K continuations on 8 PG19 documents with $K = 512$ and $T_r \in \{0, 32, 128, 512\}$. Panel (a) uses shared teacher-forced tokens; panel (b) compares autoregressive ReTopK with an Exact Top- K shadow on the same generated prefix.

Panel (a) shows that quality error remains bounded through 8K: ΔPPL is within 2.08% for all settings. More frequent refresh improves the 8K endpoint from 2.02% without refresh to 1.73% at the default $T_r = 128$ and 1.33% at $T_r = 32$, while the no-refresh setting remains stable. Panel (b) yields 0.010–0.013 mean KL up to T and 97.2–97.6% Top-1 agreement under autoregressive feedback, with no late-stage increase in error.

Kernel efficiency analysis. Table 4 reports five cumulative stages under a fixed input and path mixture. L1 separates retrieval and sparse output; L2 fuses cache search with support union and adds indexed candidate scoring; L3 adds exact sorting and deduplication while fusing selection, sparse output, and cache update; L4 reuses per-layer workspaces; and L5 fuses candidate construction with output. Relative to L1, L5 achieves 2.57 $\times$ and 2.29 $\times$ speedups at 64K and 128K, respectively.

Figure 9 decomposes attention latency across context lengths. Full Attention remains scan-dominated at 97–99%, while QK scoring and Top- K selection account for an increasing 87–97% of Exact Top- K latency, driven primarily by QK and consistent with Figure 1. ReTopK’s reuse-path cost remains nearly constant; its growth arises from full-history QK on fallback and refresh steps. This growth does not arise from the path mix: reuse rises from 87.5% to 88.9%, while low-similarity fallback falls from 11.9% to 10.5%. Instead, the per-exact-path cost grows with context length, in-

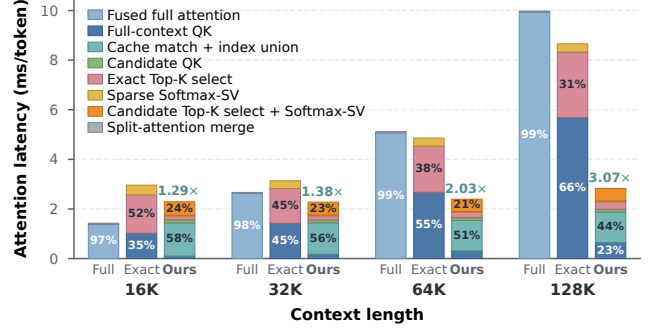


Figure 9: Attention latency composition across context lengths ($K = 512$); labels above ReTopK denote speedup over Exact Top- K .

Ctx.	GPU	Full	Exact	ReTopK	Non-attn.	Spd.
128K	1	30.30	28.88	25.63	70.1%	1.13×
256K	1	40.26	36.62	27.74	55.3%	1.32×
512K	2	60.51	51.48	31.89	39.4%	1.61×
1M	2	96.20	88.81	38.59	22.9%	2.30×
2M	4	176.22	157.28	52.72	13.0%	2.98×
3M	8	247.85	224.00	67.02	9.2%	3.34×
4M	8	325.11	290.46	81.51	7.1%	3.56×
5M	8	399.81	357.10	95.71	5.8%	3.73×

Table 5: End-to-end decoding latency (ms/token) across context lengths ($K = 512$).

creasing ReTopK’s speedup over Exact Top- K from 1.29 $\times$ to 3.07 $\times$.

End-to-end long-context scaling. We measure matched end-to-end decoding on Qwen2.5-7B-Instruct-1M, using the same GPU layout across methods and a fixed ReTopK path mixture across lengths. Timings exclude prefill, setup, and compilation.

Table 5 separates attention gains from shared decoder work. Non-attn. denotes shared latency as a fraction of Exact Top- K latency, and Spd. is relative to Exact Top- K . As the Non-attn. share decreases from 70.1% at 128K to 22.9% at 1M and 5.8% at 5M, ReTopK’s speedup increases from 1.13 $\times$ to 2.30 $\times$ and 3.73 $\times$, respectively. The end-to-end implementation evaluates the full $RK + W = 2080$ candidate capacity without truncation.

Conclusion

We introduced ReTopK, a training-free recall-before-rerank method that constructs candidate sets from cached query-support pairs, reranks them with current-query scores, and uses similarity fallback and periodic refresh for reliability. Across long-context language modeling and retrieval tasks, ReTopK preserves the quality of Exact Top- K while accelerating dynamic Top- K attention by 3.07 $\times$ at 128K with a 0.50% PPL increase. These results show that retrieval-decision reuse reduces full-history index-discovery cost without evicting KV entries.

References

- Bai, Y.; Dong, Q.; Jiang, T.; Lv, X.; Du, Z.; Zeng, A.; Tang, J.; and Li, J. 2026. IndexCache: Accelerating Sparse Attention via Cross-Layer Index Reuse. *arXiv preprint arXiv:2603.12201*.
- Bai, Y.; Lv, X.; Zhang, J.; Lyu, H.; Tang, J.; Huang, Z.; Du, Z.; Liu, X.; Zeng, A.; Hou, L.; Dong, Y.; Tang, J.; and Li, J. 2024. LongBench: A Bilingual, Multitask Benchmark for Long Context Understanding. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 3119–3137. Association for Computational Linguistics.
- Cai, Z.; Zhang, Y.; Gao, B.; Liu, Y.; Li, Y.; Liu, T.; Lu, K.; Xiong, W.; Dong, Y.; Hu, J.; and Xiao, W. 2025. PyramidKV: Dynamic KV Cache Compression Based on Pyramidal Information Funneling. In *Second Conference on Language Modeling*.
- Dao, T. 2024. FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning. In *International Conference on Learning Representations*.
- Dao, T.; Fu, D. Y.; Ermon, S.; Rudra, A.; and Ré, C. 2022. FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness. In *Advances in Neural Information Processing Systems*, volume 35.
- Deshmukh, D.; Goyal, S.; Kwatra, N.; and Ramjee, R. 2025. Cascade: A Practical Sparse Attention Method for Long-Context LLM Inference. *arXiv preprint arXiv:2512.16391*.
- Gao, Y.; Zeng, Z.; Du, D.; Cao, S.; Zhou, P.; Qi, J.; Lai, J.; So, H.; Cao, T.; Yang, F.; and Yang, M. 2025. SeerAttention: Self-distilled Attention Gating for Efficient Long-context Prefilling. In *Advances in Neural Information Processing Systems*, volume 38.
- Grattafiori, A.; et al. 2024. The Llama 3 Herd of Models. *arXiv preprint arXiv:2407.21783*.
- Hooper, C. R. C.; Kim, S.; Mohammadzadeh, H.; Maheswaran, M.; Zhao, S.; Paik, J.; Mahoney, M. W.; Keutzer, K.; and Gholami, A. 2025. Squeezed Attention: Accelerating Long Context Length LLM Inference. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 32631–32652. Association for Computational Linguistics.
- Hsieh, C.-P.; Sun, S.; Krizan, S.; Acharya, S.; Rekesh, D.; Jia, F.; Zhang, Y.; and Ginsburg, B. 2024. RULER: What’s the Real Context Size of Your Long-Context Language Models? In *First Conference on Language Modeling*.
- Jiang, H.; Li, Y.; Zhang, C.; Wu, Q.; Luo, X.; Ahn, S.; Han, Z.; Abdi, A. H.; Li, D.; Lin, C.-Y.; Yang, Y.; and Qiu, L. 2024. MInference 1.0: Accelerating Pre-filling for Long-Context LLMs via Dynamic Sparse Attention. In *Advances in Neural Information Processing Systems*, volume 37.
- Li, Y.; Huang, Y.; Yang, B.; Venkitesh, B.; Locatelli, A.; Ye, H.; Cai, T.; Lewis, P.; and Chen, D. 2024. SnapKV: LLM Knows What You Are Looking for Before Generation. In *Advances in Neural Information Processing Systems*, volume 37.
- Liu, D.; Chen, M.; Lu, B.; Jiang, H.; Han, Z.; Zhang, Q.; Chen, Q.; Zhang, C.; Ding, B.; Zhang, K.; Chen, C.; Yang, F.; Yang, Y.; and Qiu, L. 2025. RetrievalAttention: Accelerating Long-Context LLM Inference via Vector Retrieval. In *Advances in Neural Information Processing Systems*, volume 38.
- Liu, Z.; Desai, A.; Liao, F.; Wang, W.; Xie, V.; Xu, Z.; Kyrillidis, A.; and Shrivastava, A. 2023. Scissorhands: Exploiting the Persistence of Importance Hypothesis for LLM KV Cache Compression at Test Time. In *Advances in Neural Information Processing Systems*, volume 36.
- Qwen; et al. 2024. Qwen2.5 Technical Report. *arXiv preprint arXiv:2412.15115*.
- Rae, J. W.; Potapenko, A.; Jayakumar, S. M.; Hillier, C.; and Lillicrap, T. P. 2020. Compressive Transformers for Long-Range Sequence Modelling. In *International Conference on Learning Representations*.
- Ribar, L.; Chelombiev, I.; Hudlasek-Galley, L.; Blake, C.; Luschi, C.; and Orr, D. 2024. SparQ Attention: Bandwidth-Efficient LLM Inference. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, 42558–42583.
- Singhania, P.; Singh, S.; He, S.; Feizi, S.; and Bhatele, A. 2024. Loki: Low-Rank Keys for Efficient Sparse Attention. In *Advances in Neural Information Processing Systems*, volume 37.
- Tang, J.; Zhao, Y.; Zhu, K.; Xiao, G.; Kasikci, B.; and Han, S. 2024. Quest: Query-Aware Sparsity for Efficient Long-Context LLM Inference. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, 47901–47911.
- Wang, T.; Rattihalli, G.; Dhakal, A.; Li, J.; Ren, Z.; Milojevic, D.; and Shanguan, L. 2026. Predict, Reuse, and Repair: Accelerating Dynamic Sparse Attention for Long-Context LLM Decoding. *arXiv preprint arXiv:2606.30389*.
- Wu, W.; Pan, Z.; Fu, K.; Wang, C.; Chen, L.; Bai, Y.; Wang, T.; Wang, Z.; and Xiong, H. 2025. TokenSelect: Efficient Long-Context Inference and Length Extrapolation for LLMs via Dynamic Token-Level KV Cache Selection. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, 21264–21281. Association for Computational Linguistics.
- Xiao, G.; Tang, J.; Zuo, J.; Guo, J.; Yang, S.; Tang, H.; Fu, Y.; and Han, S. 2025. DuoAttention: Efficient Long-Context LLM Inference with Retrieval and Streaming Heads. In *International Conference on Learning Representations*.
- Xiao, G.; Tian, Y.; Chen, B.; Han, S.; and Lewis, M. 2024. Efficient Streaming Language Models with Attention Sinks. In *International Conference on Learning Representations*.
- Yang, A.; Yu, B.; Li, C.; Liu, D.; Huang, F.; Huang, H.; Jiang, J.; Tu, J.; Zhang, J.; Zhou, J.; Lin, J.; Dang, K.; Yang, K.; Yu, L.; Li, M.; Sun, M.; Zhu, Q.; Men, R.; He, T.; Xu, W.; Yin, W.; Yu, W.; Qiu, X.; Ren, X.; Yang, X.; Li, Y.; Xu, Z.; and Zhang, Z. 2025. Qwen2.5-1M Technical Report. *arXiv preprint arXiv:2501.15383*.
- Zhang, Z.; Sheng, Y.; Zhou, T.; Chen, T.; Zheng, L.; Cai, R.; Song, Z.; Tian, Y.; Ré, C.; Barrett, C.; Wang, Z.; and Chen,

B. 2023. H2O: Heavy-Hitter Oracle for Efficient Generative Inference of Large Language Models. In *Advances in Neural Information Processing Systems*, volume 36.